Kubernetes Pod OOMKilled Resolution SOP

Objective

Reduce memory-related pod termination when containers are killed due to memory limits or application memory leaks.

Scope

Applicable to Kubernetes workloads, platform operations, DevOps teams, SRE teams, application teams, and synthetic incident workflows across development, staging, and production-like environments.

Pre-requisites

1. Access to Kubernetes cluster using kubectl and the correct namespace context.
2. Access to application, pod, deployment, service, ingress, and node logs where applicable.
3. Access to monitoring dashboards such as Grafana, Prometheus, cloud monitoring, or container insights.
4. Permission to inspect manifests, secrets, configmaps, RBAC bindings, PVCs, and rollout history.
5. Escalation contact for DevOps, platform, infrastructure, network, and application teams.

Incident Metadata

Field	Value
Ticket ID	T039
Issue	Kubernetes pod OOMKilled
Category	Performance
Service	Kubernetes
Severity	High
Status	Resolved
Description	Pod terminated due to memory limits
Expected Resolution	Increase memory limits or optimize app

Procedure

1. Confirm the pod termination reason is OOMKilled and capture exit code, restart count, and memory limit.
2. Review container memory usage around the incident from monitoring dashboards.
3. Compare memory requests and limits with observed baseline and peak usage.
4. Check application logs for large payloads, memory leaks, cache growth, or batch processing spikes.
5. Validate whether recent code or traffic changes increased memory use.
6. Increase memory limit if usage is expected, or optimize the application if usage is abnormal.
7. Redeploy and monitor memory working set, restarts, and latency.
8. Document root cause and final resource settings in the ticket.

Common Commands

```
kubectl config current-context
kubectl get pods -n <namespace> -o wide
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --tail=100
kubectl get events -n <namespace> --sort-by=.lastTimestamp
kubectl get deploy,svc,ingress,pvc -n <namespace>
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
Pod OOMKilled	Memory limit too low	Increase memory limit after validation
Repeated OOM	Memory leak	Profile and fix application
OOM during batch job	Large payload processing	Chunk processing or increase resources
Node memory pressure	Cluster resource shortage	Scale nodes or reschedule workloads

Best Practices

1. Set memory limits using production usage data.
2. Alert on high memory utilization before OOM.
3. Profile apps with repeated OOMKilled events.
4. Avoid unbounded in-memory queues and caches.

References

Kubernetes Docs | Internal Tickets CSV | Cluster Events | Monitoring Dashboards | Application Logs | Platform Runbook